

Homework 3

Gabriela Rubio

February 23, 2026

1. (a) **Algorithm Description:** The algorithm uses the Prune and Search technique, derived from BinarySearch. We look at the middle element and its neighbors to decide if the peak is on the right or left side of the array. If the element is greater than both its neighbors, we have found the peak and can return it. Otherwise, we recurse on the side that will contain the peak.

- (b) **Pseudocode:**

```
Algorithm FindPeak(A, i, j)
Input: An unimodal array A with distinct elements,
       a starting index i, an ending index j
Output: The peak element

if i = j:
    return i

mid = (i + j)/2

if A[mid-1] < A[mid] AND A[mid+1] < A[mid]:
    return mid

else if A[mid] > A[mid + 1]
    return FindPeak(A, i, mid)

else:
    return FindPeak(A, mid + 1, j)
```

- (c) **Correctness:** The algorithm is correct because the base case is one single element, which will be the peak since there are no elements surrounding it. We divide the array always in the middle, and compare it to its surrounding elements. If the element is bigger to both elements, then we can safely assume this is the peak and return it. If the element is bigger than the element to its left but not the one to its right, then the peak should be at the at the right of the array and the other side can be safely discarded since the array is unimodally sorted. The opposite is true.

- (d) **Time Analysis:**

```
if i = j:                                // Base case, O(1)
    return i

mid = (i + j)/2                            // O(1)

if A[mid-1] < A[mid] AND A[mid+1] < A[mid]:
    return mid                            // O(1)

else if A[mid] > A[mid + 1]
    return FindPeak(A, i, mid)            // T(n/2)

else:
    return FindPeak(A, mid + 1, j) // T(n/2)
```

Our time function is $T(\frac{n}{2}) + O(1)$. We can use master theorem for this recurrence.

$$a = 1, b = 2, f(n) = 1$$

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

$$O(1) = \Theta(1)$$

This is case 2 of the Master Theorem, so $T(n) = \Theta(n^{\log_b a} \times \log n) = \Theta(1 \times \log n) = \Theta(\log n)$

Our time complexity is $O(\log n)$.

2. Let's try it out. We would have $\frac{n}{7}$ groups, and our median of medians would have to be roughly in the middle of our medians array and have half of the elements be smaller or equal to it, which would be $\frac{n}{14}$.

Each of these medians has, in their respective groups, 3 elements that are smaller than it, which would mean that in subarray we have at least $\frac{4n}{14}$ elements, and in the other we would have $n - \frac{4n}{14} = \frac{10n}{14} = \frac{5n}{7}$ elements, and the opposite be true.

Then, our recurrence would be $T(n) = T(\frac{5n}{7}) + T(\frac{n}{7}) + O(n)$

We can use guess and verify to check the time complexity of this algorithm. My guess is $T(n) = O(n)$, so that would mean that $T(n) \leq c \times n$ for some constant c .

Assuming that:

$$T(\frac{5n}{7}) \leq c \times \frac{5n}{7} \quad T(\frac{n}{7}) \leq c \times \frac{n}{7}$$

Then:

$$T(n) = c \times 5n/7 + c \times \frac{n}{7} + n$$

$$= c(\frac{5n}{7} + \frac{n}{7}) + n$$

$$c(\frac{6n}{7}) + n \leq c \times n$$

$$n \leq c \times n - c(\frac{6n}{7})$$

$$n \leq cn(1 - \frac{6}{7})$$

$$n \leq \frac{cn}{7}$$

$$1 \leq \frac{c}{7}$$

$$c \geq 7$$

Which means $T(n) = O(n)$, so the algorithm would still work as expected.

3. (a) **Algorithm Description:** This algorithm uses the Prune and Search technique to find a y-coordinate for the main pipeline. The best location would be the median of all y-coordinates of the n wells, since the median will minimize the sum of the absolute distances. We get all y-coordinates into an array Y , and then run the Selection algorithm on Y to find its median in $O(n)$ time.

(b) **Pseudocode:**

```

Algorithm OptimalPipeline(P)
Input: An set of  $n$  wells  $P$  with coordinates  $(x, y)$ 
Output: the optimal y-coordinate for the main pipeline

 $Y = \{\}$ 
for each well  $p$  in  $P$ :
    add  $y$  to  $Y$ 

return Selection( $Y, n/2$ )
    
```

- (c) **Correctness:** The algorithm is correct because it merely calls the studied Selection algorithm to return the median ($n/2$ th element) of our Y coordinates array. The median will optimize the location of the pipeline, since it will be the point that is the closest possible to all the elements—moving from the median will make it so some of the wells get unnecessarily farther from the pipeline. Since there are equal numbers of wells on each side at the median, the total distance gets increased. Therefore, the median is the optimal location.

(d) **Time Analysis:**

```

Y = {}                                // 0(1)
for each well p in P:                 // 0(n)
    add y to Y

return Selection(Y, n/2)               // 0(n)

```

Since both looping through the set and calling Selection take $O(n)$ time, and the initializing only takes $O(1)$, our time function is $T(n) = O(n) + O(n) + O(1) = O(n)$. Our time complexity is $O(n)$.

4. (a) **Algorithm Description:** This algorithm uses the Prune and Search technique to find a y-coordinate for the main pipeline. The best location would be a weighted median where the total weight of all wells below it is at most half of the total sum of the weights, and the total weight of all wells above it is at most half of the same sum. We get the y-coordinates and their weights into an array, compute Z as half the total weight, and then call WeightedSelection to find the weighted median.

(b) **Pseudocode:**

```

Algorithm OptimalWeightedPipeline(P)
Input: A map of n wells P with coordinates (x, y) and weights w
Output: the optimal y-coordinate for the main pipeline

Y = {}
weight = 0
for each well p in P:
    add y and w to Y
    weight = weight + w

return WeightedSelect(Y, weight/2)

```

- (c) **Correctness:** The algorithm is correct because it merely calls the studied WeightedSelect algorithm to return the weighted median of our Y coordinates + Weighted array. The weighted median will optimize the location of the pipeline, since it will be the point that is the closest possible to all the elements, prioritizing the elements with highest weight—moving from the median will make it so some of the wells get unnecessarily farther from the pipeline. Since the total weight on each side is at most half the total weight at the weighted median, moving in any direction increases the total weighted distance. Therefore, the weighted median is the optimal location.

(d) **Time Analysis:**

```

Y = {}                                // 0(1)
weight = 0                            // 0(1)
for each well p in P:                 // 0(n)
    add y and w to Y
    weight = weight + w

return WeightedSelect(Y, weight/2)    // 0(n)

```

Since both looping through the set and calling WeightedSelect take $O(n)$ time, and the initializing only takes $O(1)$, our time function is $T(n) = O(n) + O(n) + O(1) = O(n)$. Our time complexity is $O(n)$.

5. (a) **Algorithm Description:** This algorithm combines the technique of Divide and Conquer and the Selection algorithm to select multiple k-th smallest elements. Instead of calling selection multiple times on the same set of elements, we partition both the numbers array and the sequence everytime we find a new kth smallest element, ensuring that each recursive call works on a smaller subset of A rather than the full array. We find the middle k value, use Selection to find the corresponding element in A as our pivot, partition A into elements

smaller and larger than the pivot, and split K accordingly. We recurse on each half until there's no more elements to be found, or our A array is sufficiently small that we can safely sort and return the elements.

(b) **Pseudocode:**

```

Algorithm MultipleSelection(A, K)
Input: An array of  $n$  numbers  $A$ , a sorted array of  $m$  integers  $K$ 
Output: An array of the  $k_i$ -th smallest numbers of  $A$ 

if  $|k| == 0$ :
    return {}

if  $|A| \leq 100$ :
    sort  $A$ 
    return  $\{A[k_1], A[k_2], \dots A[k_m]\}$ 

index =  $K[|K|/2]$ 
pivot = Selection( $A$ , index)

 $A_1 = \{\}$ 
 $A_2 = \{\}$ 

for each element  $n$  in  $A$ :
    if  $n < \text{pivot}$ :
        add  $n$  to  $A_1$ 
    if  $n > \text{pivot}$ :
        add  $n$  to  $A_2$ 

 $K_1 = \{\}$ 
 $K_2 = \{\}$ 

for each element  $i$  in  $K$ :
    if  $i < \text{index}$ :
        add  $i$  to  $K_1$ 
    if  $i > \text{index}$ :
        add  $(k_i - |A_1| - 1)$  to  $K_2$ 

leftSide = MultipleSelection( $A_1$ ,  $K_1$ )
rightSide = MultipleSelection( $A_2$ ,  $K_2$ )

set = {}

for each number  $n$  in leftSide:
    add number to set
add pivot to set
for each number  $n$  in rightSide:
    add number to set

return set

```

- (c) **Correctness:** The algorithm is correct because, starting with the base cases: if there's no more elements to be found, we can safely return an empty set, and if our array is small enough, sorting and returning the elements we need will always be correct. After choosing the middle index in K , Selection returns the correct pivot element. It is safe to partition A around this pivot because all elements in A_1 are smaller than the pivot and all elements in A_2 are larger. Similarly, all k values in K_1 are smaller than the middle index, meaning their answers must be in A_1 , and all k values in K_2 are larger, meaning their answers must be in A_2 . We adjust K_2 values by subtracting $|A_1| + 1$ because we are removing A_1 and the

pivot from consideration, so the ranks shift accordingly. Finally, combining leftSide + pivot + rightSide gives the correct ordered result because everything in leftSide is smaller than the pivot and everything in rightSide is larger.

(d) **Time Analysis:**

```

if |k| == 0:                                // O(1)
    return {}

if |A| <= 100:                              // O(1)
    sort A
    return {A[k1], A[k2], ... A[km]}

index = K[|K|/2]                            // O(1)
pivot = Selection(A, index)                 // O(n)

A1 = {}
A2 = {}

for each element n in A:                   // O(n)
    if n < pivot:
        add n to A1
    if n > pivot:
        add n to A2

K1 = {}
K2 = {}

for each element i in K:                   // O(m)
    if i < index:
        add i to K1
    if i > index:
        add (ki - |A1| - 1) to K2

leftSide = MultipleSelection(A1, K1) // T(|A1|, m/2)
rightSide = MultipleSelection(A2, K2) // T(|A2|, m/2)

set = {}

for each number n in leftSide:             // O(m)
    add number to set
add pivot to set
for each number n in rightSide:
    add number to set

return set

```

Both our base cases are $O(1)$ (because 100 is a fixed number), and looping through our A is $O(n)$, while looping through or K is $O(m)$, and recursing on both sides is $T(|A1|, \frac{m}{2}) + T(|A2|, \frac{m}{2})$. That gives us the time function $T(n, m) = T(|A1|, \frac{m}{2}) + T(|A2|, \frac{m}{2}) + O(n)$.

I will use guess and verify for this.

I want to prove that $T(n, m) = O(n \log m)$.

So, $T(n, m) \leq c * n \log m$, for a $c > 0$

I'll assume that

$$T(|A1|, \frac{m}{2}) \leq c * |A1| * \log(\frac{m}{2})$$

and

$$T(|A2|, \frac{m}{2}) \leq c * |A2| * \log(\frac{m}{2})$$

Plug that in:

$$c * |A1| * \log(\frac{m}{2}) + c * |A2| * \log(\frac{m}{2}) + n$$

$$c * \log(\frac{m}{2}) * (|A1| + |A2|) + n$$

$$c * \log(\frac{m}{2}) * (n - 1) + n \leq c * n \log m$$

$$c * (\log m - 1) * (n - 1) + n \leq c * n \log m$$

$$c * (n \log m - n - \log m + 1) + n \leq c * n \log m$$

$$(c * n \log m) - cn - (c * \log m) + c + n \leq c * n \log m$$

$$-cn - c \log m + c + n \leq 0$$

$$n \leq cn + c \log m - c$$

$$n \leq c(n + \log m - 1)$$

$$\frac{n}{n + \log m - 1} \leq c$$

$$c \geq \frac{n}{n + \log m - 1}$$

Since $\frac{n}{n + \log m - 1} < 1$ for all n and m , any constant $c \geq 1$ satisfies the inequality. Therefore $T(n, m) = O(n \log m)$.